Devoir surveillé d'informatique

Consignes

- Les programmes demandés doivent être écrits en C ou en OCaml suivant l'exercice. Dans le cas du C, on suppose que les bibliothèques standard usuelles (`<stdio.h>`, `<stdlib.h>`, `<stdbool.h>`) sont déjà importées.
- On pourra toujours librement utiliser une fonction demandée à une question précédente même si cette question n'a pas été traitée.
- Veillez à présenter vos idées et vos réponses partielles même si vous ne trouvez pas la solution complète à une question.
- La clarté et la lisibilité de la rédaction et des programmes sont des éléments de notation.

❑ Exercice 1 : *Multiplication à la russe*

On donne ci-dessous un algorithme appelé *multiplication à la russe* qui prend en entrée deux nombres entiers positifs et renvoie leur produit en effectuant uniquement des divisions et multiplications par 2 ainsi que des additions et soustractions :

Algorithme : Multiplication à la russe

Entrées : $a \in \mathbb{N}, b \in \mathbb{N}$

Sorties : $a \times b$

```
r ← 0
tant que a ≠ 0 faire
    si a est impair alors
        r ← r + b
    fin
    a ← ⌊a/2⌋
    b ← b * 2
fin
return r
```

Q1– Donner les valeurs successives prises par les variables a, b et r si on fait fonctionner cet algorithme avec $a = 57$ et $b = 68$ en recopiant et en complétant le tableau suivant (on donne : $57 \times 68 = 3876$) :

	a	b	r
valeurs initiales	57	68	0
après un tour de boucle	...	...	...
après deux tours de boucle	...	...	...
après trois tours de boucle	...	...	...
après quatre tours de boucle	...	...	...
après cinq tours de boucle	...	...	...
après six tours de boucle	...	...	...

Q2– Donner une implémentation de cet algorithme en langage C sous la forme d'une fonction `russe` de signature `int russe(int a, int b)`.

Q3– Prouver que cet algorithme termine.

Q4– Prouver que cet algorithme est correct. En notant a_0 (resp. b_0) la valeur initiale de a (resp. b), on pourra prouver l'invariant $r + ab = a_0 b_0$.

Q5– Pour a et b deux entiers naturels, exprimer le produit ab en fonction du produit de $\lfloor \frac{a}{2} \rfloor$ par $2b$ (distinguer les cas pair et impair). En déduire une version récursive de l'algorithme de multiplication à la russe et en donner une implémentation en langage C.

Q6– Écrire une version récursive de la multiplication à la russe en OCaml.

❑ Exercice 2 : *Quelques expressions en OCaml*

Pour chacune des expressions ci-dessous, indiquer son type et sa valeur lorsqu'elle s'évalue sans erreur. Sinon indiquer la cause de l'erreur rencontrée.

Q7– `let n = 24 mod 7;;`

Q8– `let perimetre = 4 *. 2.5;;`

Q9– `let v = 2.0**10;;`

Q10– `let at = '@' in print_char at;;`

Q11– `let coucou = let message = "Bonjour " + "tout le monde" in print_string message;;`

Q12– `let peri = let cote = 5 in 4*cote;;`

Q13– `let langage = "OCaml" in langage.[1];;`

Q14– `let x = 42 in (x / 10 > 4) && (x <= 21 || x>=42) ;;`

Q15– `let k = if 2=1+1 then 'A' else 'B';;`

Q16– `let x = 10 in if x > 0 then 1 else 0.0;;`

Q17– `let x = 5 in if x > 0 then x + 1;;`

Q18– `let rec fact n = if n=0 then 1 else n* fact (n-1);;`

Q19– `let double x = 2 * x in double -3;;`

Q20– `let l = [1; 2] :: [3; 4];;`

❑ **Exercice 3** : *Couleurs au format RVB*

En informatique, une couleur au format RVB (Rouge, Vert, Bleu) est représentée par trois composantes entières comprises entre 0 et 255. En langage C, on choisit de représenter chacune de ces composantes par le type `uint8_t` (entier non signé sur 8 bits, défini dans `<stdint.h>`) et on définit le type structuré suivant :

```c
struct couleur_s
{
    uint8_t r;
    uint8_t v;
    uint8_t b;
};
typedef struct couleur_s couleur;
```

Q21– Sur le Web (HTML/CSS), une couleur RVB est souvent notée en hexadécimal sur 6 chiffres sous la forme `#RRVVBB`, où `RR`, `VV` et `BB` sont les écritures en base 16 sur deux chiffres des composantes `r`, `v` et `b`. Donner les valeurs décimales des champs `r`, `v` et `b` de la couleur codée par `#3A80FF`.

Q22– On considère la couleur `c` initialisée par `couleur c = {.r = 25, .v = 200, .b = 12};`. Donner l'écriture binaire sur 8 bits de chacune de ses trois composantes, puis en déduire le code hexadécimal `#RRVVBB` de cette couleur.

Q23– La fonction `couleur melanger(couleur c1, couleur c2)` ci-dessous doit renvoyer une nouvelle couleur dont chaque composante est la moyenne entière des composantes correspondantes de `c1` et `c2`. Cette fonction est-elle correcte ? Expliquer et proposer une correction si ce n'est pas le cas.

```c
couleur melanger(couleur c1, couleur c2)
{
    couleur m;
    uint8_t sr = c1.r + c2.r;
    uint8_t sv = c1.v + c2.v;
    uint8_t sb = c1.b + c2.b;
    m.r = sr / 2;
    m.v = sv / 2;
    m.b = sb / 2;
    return m;
}
```

Q24– Le négatif d'une couleur (r, v, b) est la couleur $(255-r, 255-v, 255-b)$. Écrire une fonction de signature `couleur negatif(couleur c)` qui renvoie le négatif de la couleur donnée en argument.

Q25– Au lieu de *renvoyer* le négatif d'une couleur, on veut écrire une fonction `transforme_negatif` qui ne renvoie rien mais transforme la couleur donnée en argument en son négatif. Expliquer pourquoi la signature de cette nouvelle fonction est différente de celle de la fonction `negatif` de la question précédente, puis écrire la fonction `transforme_negatif`.

Q26– Écrire une fonction `couleur* palette(int n)` qui alloue dynamiquement et renvoie une palette de `n` couleurs toutes tirées au sort. On rappelle que la fonction `rand()` du langage C (du module `<stdlib.h>`) renvoie un entier compris entre 0 et une constante `RAND_MAX`.

Q27– Écrire une fonction de signature `bool depuis_hexa(char *chaine, couleur *res)` qui prend en argument une chaîne de caractères `chaine` et un pointeur `res` vers une couleur. La fonction doit vérifier que `chaine` est bien au format `"#RRVVBB"` (c'est-à-dire qu'elle comporte exactement 7 caractères, commence par `'#'` et que les 6 caractères suivants sont des chiffres hexadécimaux valides compris entre `'0'` et `'9'` ou entre `'A'` et `'F'`). Si le format est valide, la fonction stocke la couleur correspondante à l'adresse pointée par `res` et renvoie `true` ; sinon, elle renvoie `false` sans modifier `*res`.
(On rappelle que dans la table ASCII, les caractères `'0'` à `'9'` ont des codes consécutifs, tout comme les caractères `'A'` à `'F'`).

❑ Exercice 4 : *Rotation droite d'un tableau*

Dans cet exercice, on ne tient pas compte des éventuels dépassements de capacité lors des additions ou soustractions sur les entiers et on cherche à écrire, *sans utiliser de variable intermédiaire*, une fonction permettant d'effectuer la rotation droite d'un tableau. Par exemple, en effectuant une rotation droite, le tableau `{2, 6, 4, 1}` devient `{1, 2, 6, 4}` : c'est-à-dire que tous les éléments sont décalés d'un cran vers la droite, sauf le dernier qui devient le premier. D'un point de vue formel, la rotation droite d'un tableau T (de taille $n \geqslant 1$) est le tableau T' tel que $T'[0] = T[n-1]$ et $T'[k] = T[k-1]$ pour tout $k \in [\![1; n-1]\!]$.

Q28– Écrire une fonction `void echange(int tab[], int i, int j)` qui prend en argument un tableau d'entiers ainsi que deux indices distincts `i` et `j` supposés valides (c'est-à-dire compris entre 0 et la taille du tableau moins 1, avec $i \neq j$) et échange les éléments situés aux indices `i` et `j` *sans utiliser de variable intermédiaire.*

Q29– Écrire une fonction de signature `void rotation(int tab[], int n)` permettant d'effectuer une rotation droite de tous les éléments d'un tableau *sans utiliser de variable intermédiaire pour stocker un élément du tableau.*

Q30– En proposant un invariant de boucle, prouver que la fonction `rotation` est correcte.